# OS-Orange 2

**Name: Tusti S**

**SRN: PES1UG24CS505**

## Phase 1:

```
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$
```

## Phase 2:

```
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$
```

```
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$ find .pes/objects -type f
.pes/objects/f8/b93f79591de4e825e2072f77789783d428dceb30c5cc2a8e7484465a0ceab9
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$ xxd .pes/objects/XX/<file> | head -20
bash: syntax error near unexpected token `|'
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$ xxd .pes/objects/f8/b93f79591de4e825e2072f77789783d428dceb30c5cc2a8e7484465a0ceab9 | head -20
00000000: 7472 6565 2034 3800 3130 3036 3434 2066  tree 48.100644 f
00000010: 696c 652e 7478 7400 aaaa aaaa aaaa aaaa  ile.txt........
00000020: aaaa aaaa aaaa aaaa aaaa aaaa aaaa aaaa  ................
00000030: aaaa aaaa aaaa aaaa                      ........
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$
```

# Phase 3:

```
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$ ./pes init
echo "hello" > a.txt
echo "world" > b.txt

./pes add a.txt b.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     a.txt
  staged:     b.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.h
  untracked:  commit.h
  untracked:  tree.c
  untracked:  tree.h
  untracked:  README.md
  untracked:  object.c
  untracked:  test_tree.c
  untracked:  pes.c
  untracked:  Makefile
  untracked:  temp.c
  untracked:  index.h
  untracked:  file2.txt
  untracked:  test_sequence.sh
  untracked:  test_objects.c
  untracked:  index.c
  untracked:  file1.txt
  untracked:  commit.c
  untracked:  temp

tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$
```

```
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776683298 6 a.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776683298 6 b.txt
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$
```

# Phase 4:

```
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$ gedit commit.c
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$ make
./pes commit -m "first commit"
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Committed: fbb55e8b500d... first commit
tusti-sindigi@tusti-sindigi-IdeaPad-Slim-5-14IMH9:~/projects/PES1UG24CS505-pes-vcs$
```

# Phase 5:

Q5.1
To implement pes checkout <branch>, first read the commit hash stored in .pes/refs/heads/<branch>. Then update .pes/HEAD to point to that branch (ref:refs/heads/<branch>). Next, load the commit object, extract its tree, and reconstruct the working directory so that all files match that tree. The index must also be updated to reflect the checked-out state. This operation is complex because it involves safely replacing files in the working directory, handling nested directories, and ensuring no user data is lost if there are local Changes.

Q5.2
To detect a dirty working directory, compare each tracked file's current state with its entry in the index. For each file, check metadata like size and modification time; if they differ, recompute the file's hash and compare it with the stored hash in the index. If the file's current content differs from the index, it is modified. Then compare the index version with the target branch's version. If both differ and the file would be overwritten, refuse checkout. This uses only the index and object store.

Q5.3
In a detached HEAD state, new commits are created normally but are not referenced by any branch. As a result, they can become unreachable once the user switches away. The commits still exist in the object store but are not part of any branch history. A user can recover them by creating a new branch pointing to the commit hash (if known) or by using logs or reflog-like mechanisms if available.

# Phase 6:

Q6.1
Start from all branch heads in .pes/refs/heads/ and perform a traversal of the commit graph. For each commit, mark it as reachable, then recursively visit its parent and its tree. From each tree, recursively visit all referenced blobs and subtrees. Use a hash set to store all reachable object IDs efficiently. After traversal, scan the object store and delete any object not in the reachable set. For a repository with 100,000 commits and 50 branches, the traversal will visit at most around 100,000 commits (since branches share history) plus associated trees and blobs, not 50 × 100,000.

Q6.2

Garbage collection is dangerous during concurrent commits because a race condition can occur where GC deletes an object that a commit is in the process of creating or about to reference. For example, a commit writes a tree and blob but has not yet updated HEAD; GC may not see these as reachable and delete them. Git avoids this by using mechanisms like temporary references, lock files, and by only collecting objects that are old and not recently modified. It also ensures that objects are fully written and referenced before they can be considered for deletion.